

Learned indexes: from positions to compressed layouts

A historical walkthrough with inspectable evidence and runnable experiments

UNDERSTAND

What changed at each stage of the literature?

RUN

Small offline experiments linked by lesson ID.

DECIDE

Agree on the next common-data reproduction milestone.

Existing work first. A new contribution comes after the baseline.

Three evidence labels — three different claims

A rerun of arithmetic is not a reproduction of a published system.

Label	What it supports	Example
PUBLISHED	What the paper reports in its setup	NFL Tables 1–2; CSV promoted keys
TEACHING DEMO	Inspect a mechanism and its objective	Two-stage fitting; virtual points
LOCAL MEASUREMENT	What tested code does on this fixture	RadixSpline-derived; custom map

No full ALEX / PGM / NFL / CSV upstream reproduction is claimed.

The history is branching, not a single leaderboard

The papers improve different parts of the same access path.

Period	Main idea	Representative approaches
2018–20	Predict positions; bound error; build cheaply	RMI → FITing-Tree → PGM / RadixSpline
2020–23	Maintain learned structure under updates	ALEX → LIPP → SALI
2021–24	Compress while retaining direct access	LA-vector → LeCo
2022–25	Change the distribution / positions	NFL → CSV
2025–26	Evaluate robustness and maintenance tails	SEA comparison / RoBin / LINE

Start with the task: exact ordered lookup

Use case: look up integer-keyed records and scan their successors.

INPUT

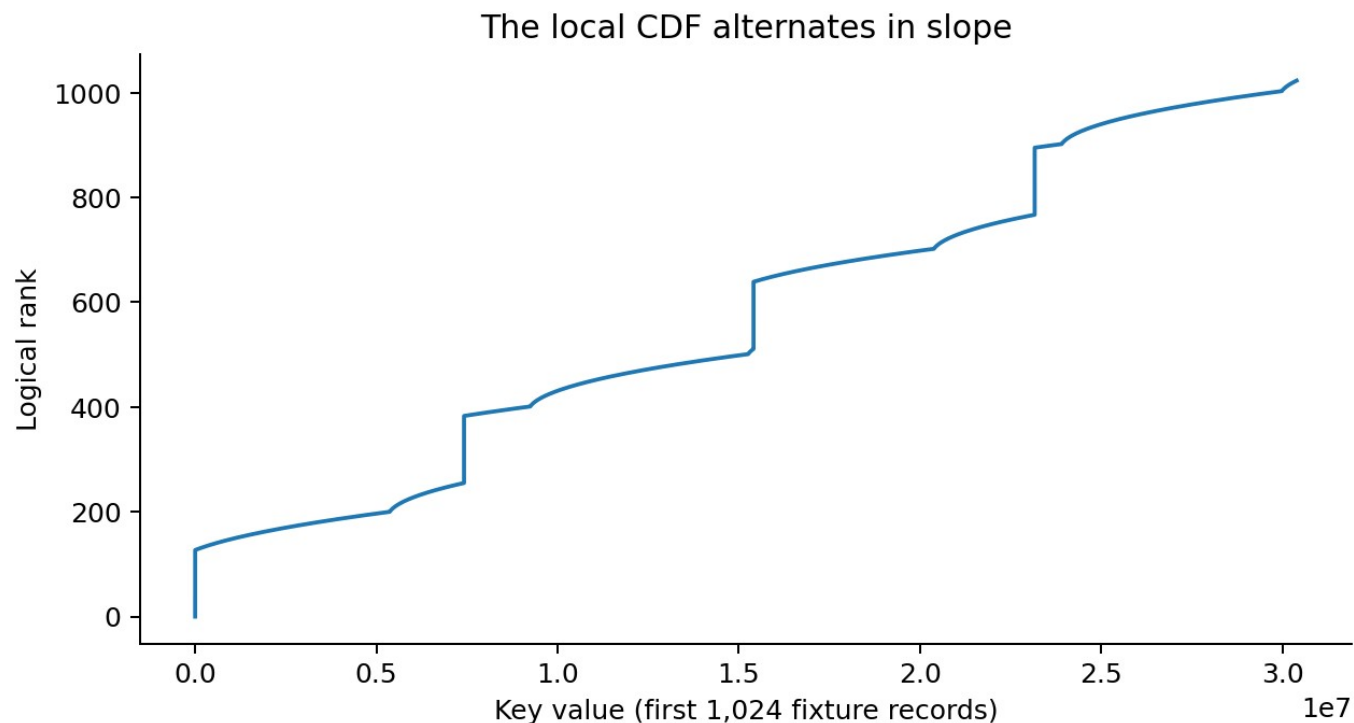
16,384 distinct sorted uint64 keys.
Alternating dense and sparse intervals.

OUTPUT

`lower_bound(k)`: the first stored key
that is at least `k`.

BASELINE

Binary search gives an exact answer
without a fitted predictor.



2018 • predict rank, then specialize

RMI introduces model-guided navigation and exact correction.

MECHANISM

A root prediction selects a local model.
That model predicts a position.

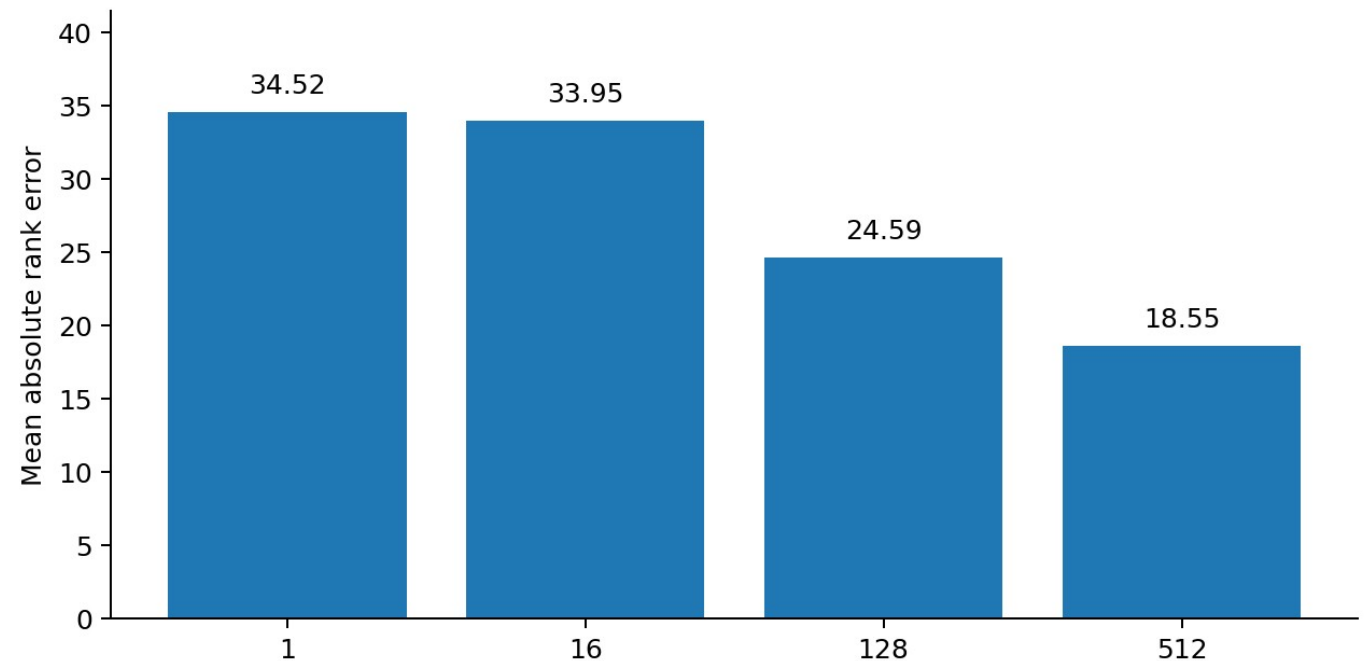
WHAT WE CHECK

20,482 exact query checks after
corrective search.

TRADE-OFF

More models can help fitting, but
consume space and add routing work.

Teaching model: effect of leaf count



L01 python3 walkthrough.py demo 01

2019 • make prediction error an explicit parameter

FITing-Tree: piecewise-linear approximation with a chosen error bound.

BEFORE

A good average fit can still leave difficult local regions.

CHANGE

Partition into linear segments that meet a specified error tolerance.

COST

Tighter tolerance may require more segments and metadata.

Prediction + bounded correction, not “the model must be exact.”

2020 • build a hierarchy of bounded models

PGM indexes the segments recursively and supplies static and dynamic variants.

MODELS OF MODELS

Each level narrows the next search; the final interval is corrected exactly.

BOUND + SPACE

Error tolerance determines the fit/space/search trade-off.

DISTINCTION

Compressing model parameters is not automatically compressing the keys.

2020 • RadixSpline makes the path concrete

One-pass spline construction + radix directory + exact bounded search.

FINAL WINDOW

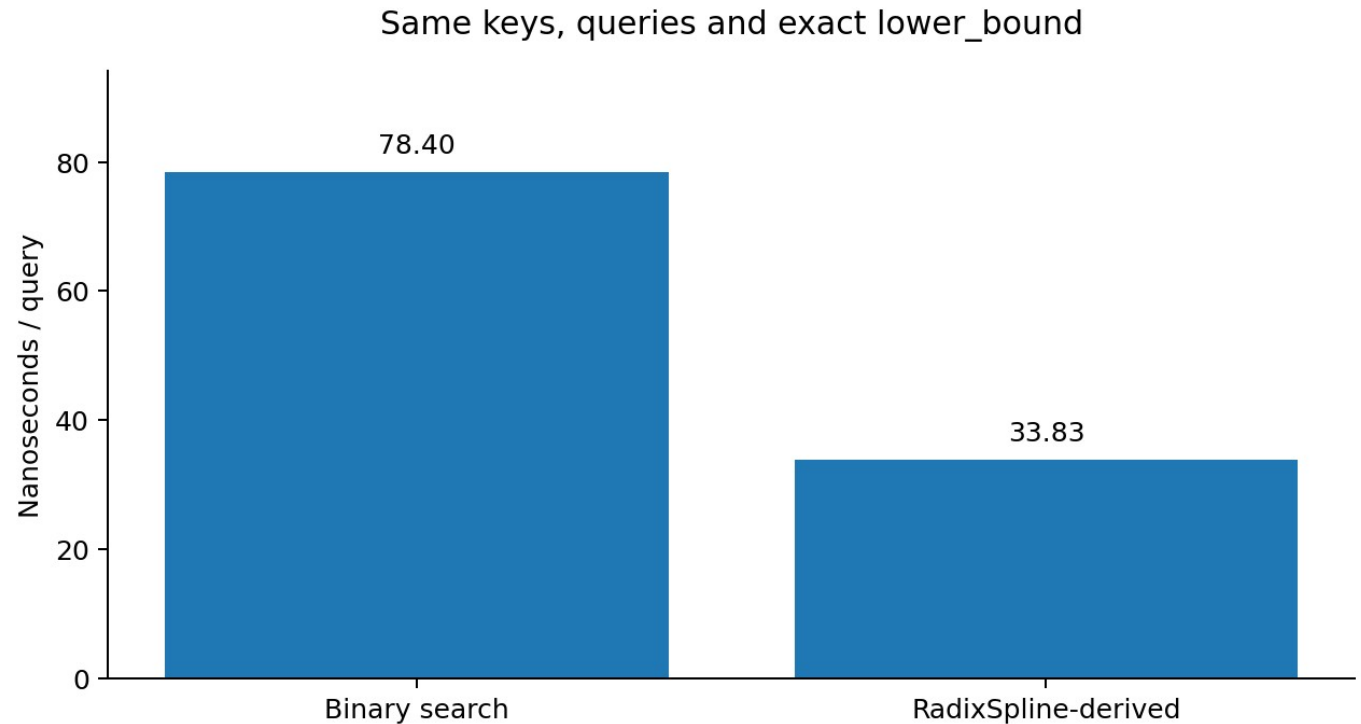
At most 66 keys in this run, with $\epsilon = 32$.

EXACTNESS

Every query in the measured trace checked against `std::lower_bound`.

SPACE SCOPE

131,072 key bytes + 5,908 auxiliary bytes; no payloads.



L02 python3 walkthrough.py demo 02

2020–21 • updates require a storage strategy

ALEX and LIPP address insertion and conflicts differently.

ALEX

Use gaps and adaptive expansion/splitting to accommodate writes.

LIPP

Use predicted placement; represent collisions through additional nodes.

The baseline must include inserts, scans, memory and maintenance — not reads alone.

2022 • NFL changes the input distribution

Transform numerical keys before building an After-Flow Learned Index.

KEY CHANGE

Original key → numerical flow →
transformed key → learned index.

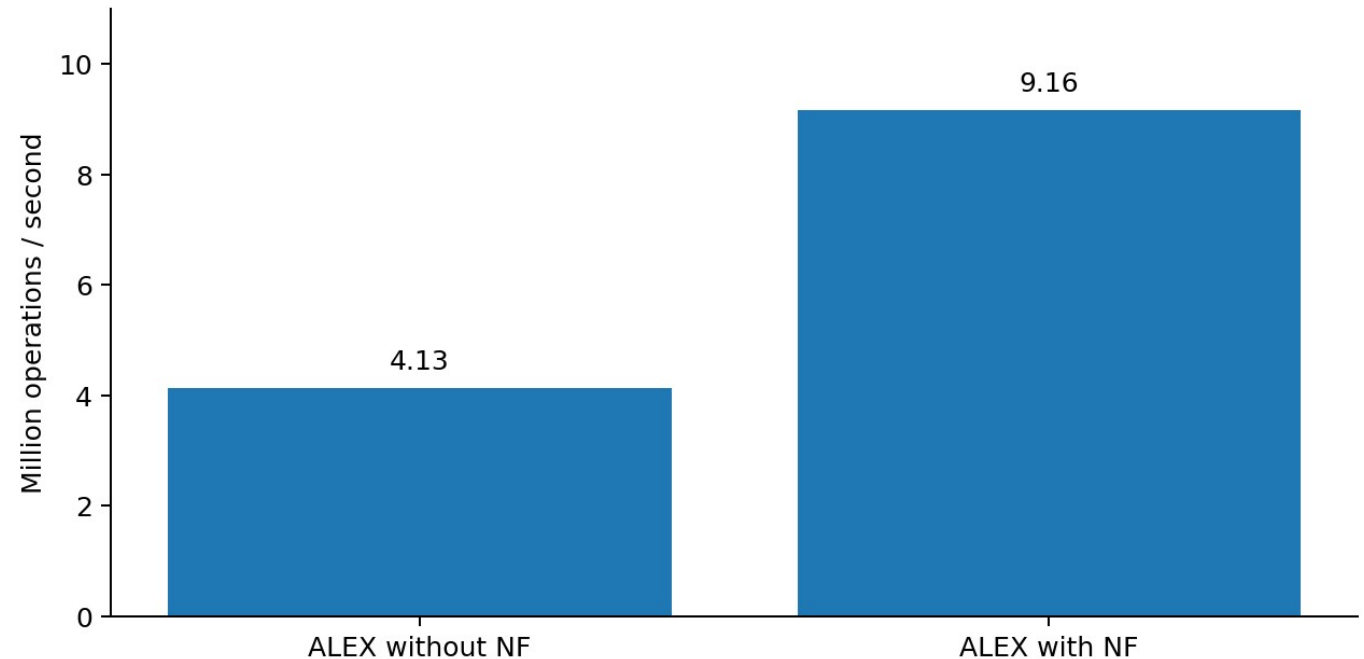
REPORTED EXAMPLE

Facebook: max tree height 11 → 3.
Throughput 4.13 → 9.16 Mops/s.

NEXT QUESTION

Does the lower search cost repay the
online transformation?

Published result: Facebook, NFL Table 1



L03 python3 walkthrough.py demo 03

The cost of a transformation is part of the result

Batching changes the per-key economics; latency definitions also matter.

SINGLE KEY

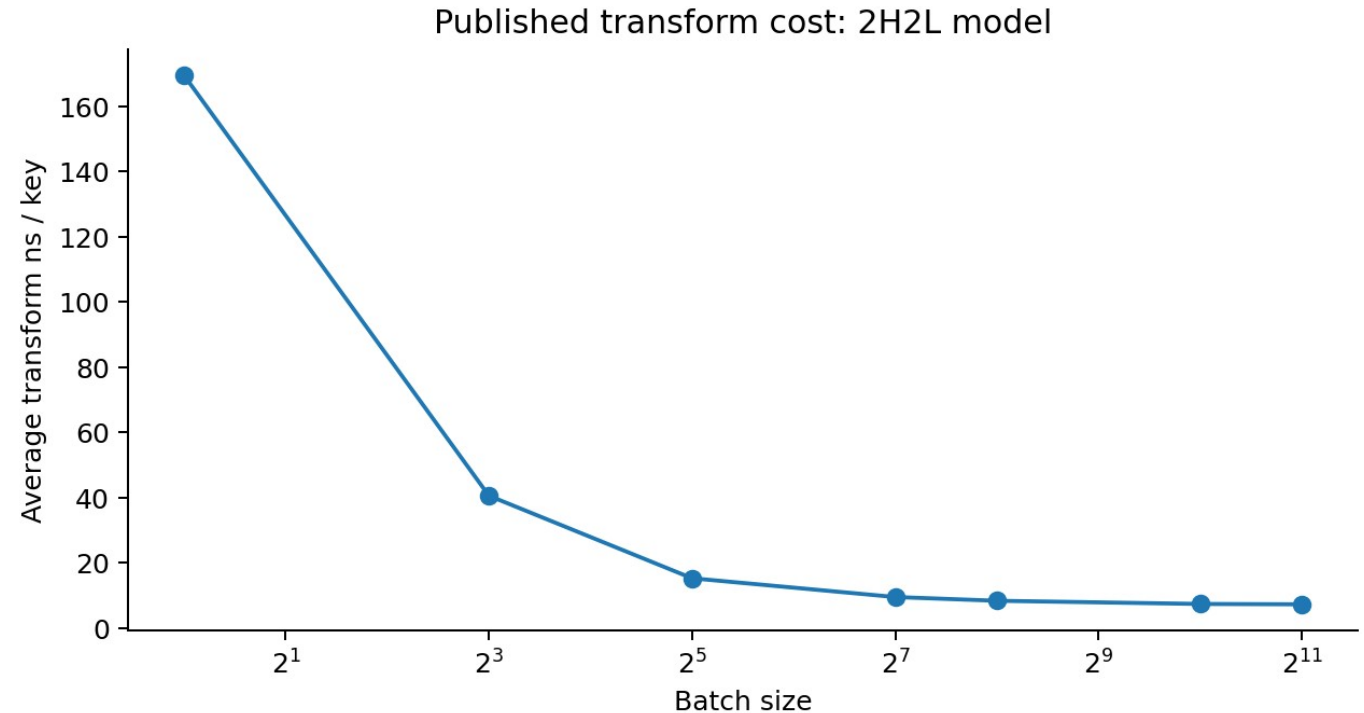
169.53 ns/key average transformation cost, reported for 2H2L.

BATCH OF 256

8.38 ns/key average, in that same table.

DO NOT CONFLATE

Batch-normalized P99 is not the P99 of individual request latency.



L03 python3 walkthrough.py demo 03

2021–24 • compress without losing efficient access

A parallel line of work: LA-vector and LeCo.

LA-VECTOR

A learned compressed representation with exact rank/select operations.

LECO

Model position → value, then store exact residuals and partition metadata.

PUBLISHED SYSTEM RESULT

LeCo reports up to 16% higher Seek throughput in its RocksDB setup.

That experiment uses 900M generated records and 64 threads — not our workload.

Run a control: compression can save space yet slow reads

Hold keys, block boundaries, query traces and exact map semantics fixed.

REPRESENTATIONS

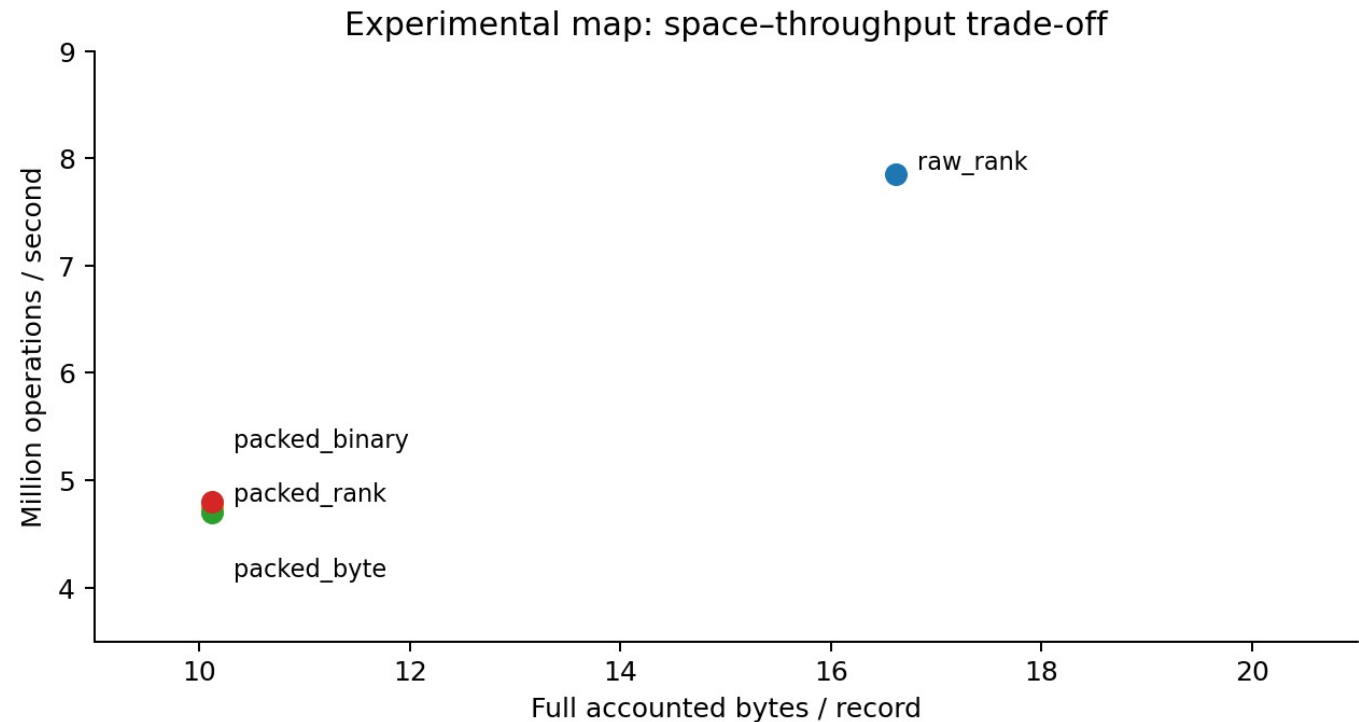
Raw, FOR, restarted delta and integer-linear residual blocks.

MATCHED COMPARISONS

Raw/rank → packed/rank → packed/byte.
Packed/binary is another control.

OBSERVATION

The packed footprint is smaller. This run does not show a throughput win.



L04 python3 walkthrough.py demo 04

CSV • change positions with virtual points

Refit models and reconstruct selected subtrees when the cost condition permits.

MECHANISM	REPORTED BENEFIT	COST TO REPORT
Insert virtual positions, refit, then potentially reduce traversal depth.	Up to 34% less query time for the selected promoted keys.	At $\alpha = 0.1$, ALEX preprocessing ranges from 889 to 2,902 seconds.

Conditional query gain $\neq$ whole-workload speedup. Preprocessing is not free.

Make virtual-point smoothing inspectable

A ten-key teaching example exposes exactly what the objective changes.

INPUT

10 explicit keys; budget = 3 virtual points. No hidden dataset.

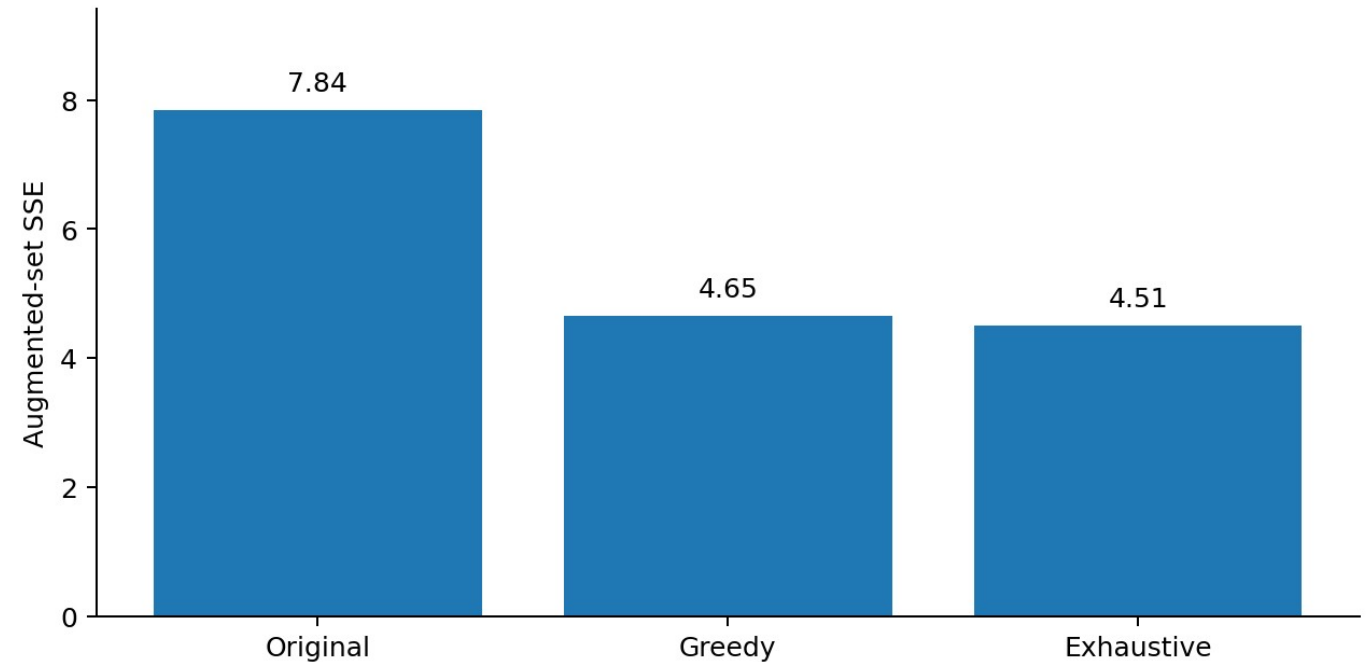
VERIFY

Exhaustive search provides a ceiling over this finite candidate set.

LIMIT

These are fitting errors, not query-time savings. The full CSV hierarchy is absent.

Independent ten-key teaching example



The project hypothesis must distinguish rank from bytes

Lossless compression changes representation, not the order of the same keys.

INVARIANT

Logical rank model unchanged across raw, packed and density-target layouts.

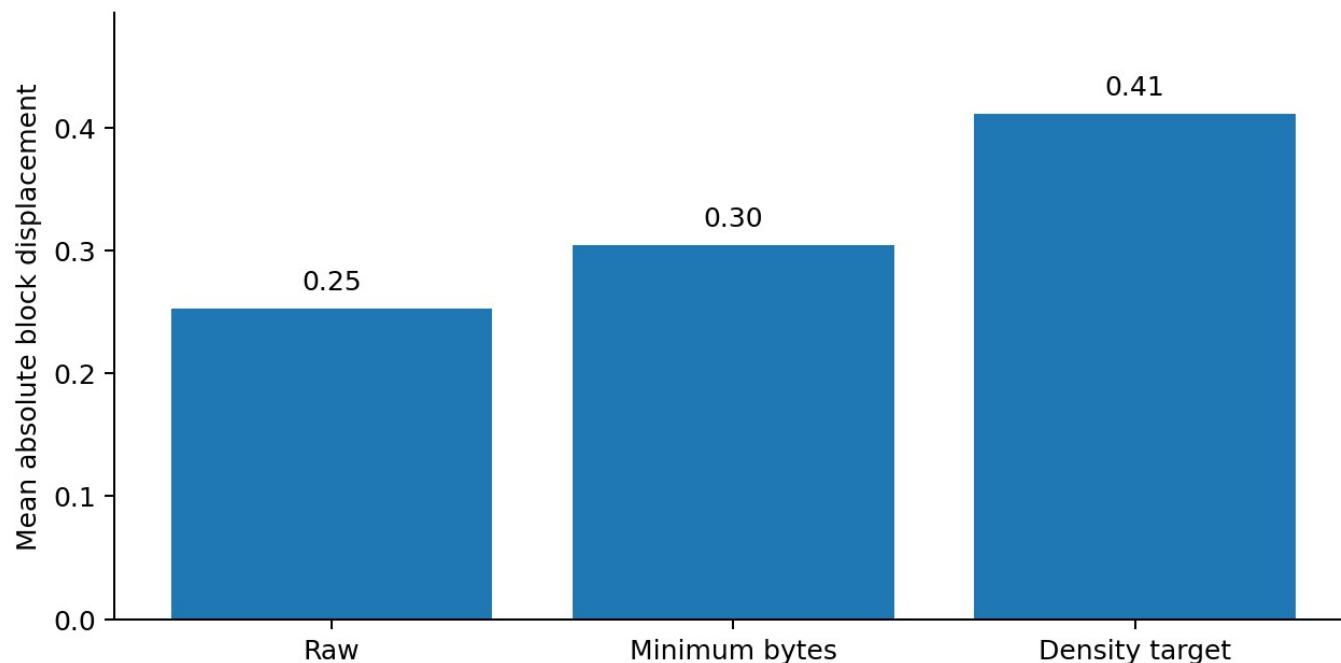
WHAT WE MEASURE

Encoded bytes, normalized fitting error, initial block displacement.

OBSERVATION

Lower byte-fitting error did not imply lower block-search error here.

A smaller byte error is not the search objective



L06 python3 walkthrough.py demo 06

Updates expose another cost: rebuilding a region

A compression benefit must survive maintenance, not just the initial build.

UPDATE PATH

Recent changes enter a sorted overlay. A full overlay triggers local rebuilding.

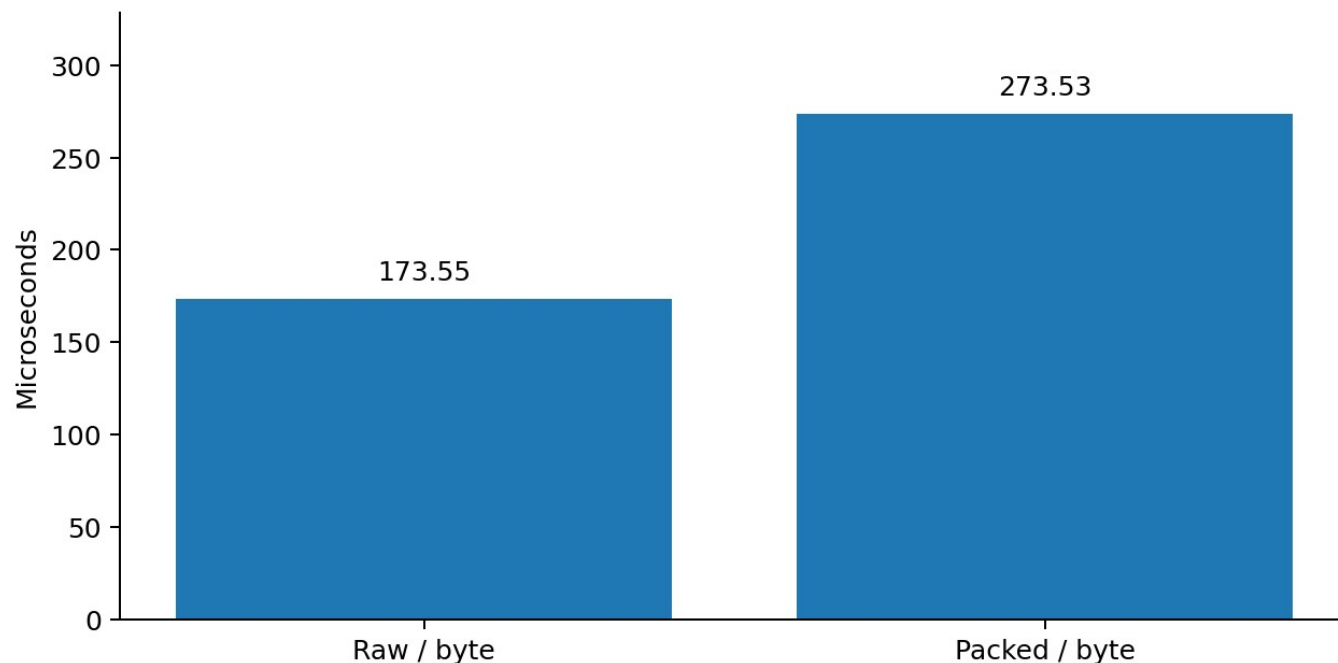
FAIR CONTROL

Raw and compressed variants both perform maintenance on the same trace.

QUESTION

Which cost comes from decoding, copying, allocation, fitting or rewriting?

Matched insert P99: synchronous maintenance



2025–26 • robustness is part of the baseline

Recent work reinforces the need to vary workloads and control maintenance tails.

STRONG CONTROLS

Compare against optimized traditional and compressed representations.

ROBUSTNESS GRID

Vary initial sampling, initial size, insertion order and domain growth.

MAINTENANCE

LINE decomposes lengthy structural changes into group-wise migrations.

Do not infer a universal ranking from a favorable read-only dataset.

What is ready — and what still needs reproduction

A transparent baseline plan is stronger than overstating coverage.

Ready now	Not yet established
Chronological deck + traceable paper evidence	Full benchmark reproduction for each paper
7 executable lessons + saved offline outputs	Authors' RMI/FITing/ALEX/PGM/NFL/CSV runs
Native source-derived RadixSpline comparison	A commit-pinned upstream parity claim
Exact custom-map compression/update controls	Improvement over SOTA or real-data robustness

Agree on the next baseline milestone

Use the history to choose experiments, not to preselect a new method.

1 · COMMON CONTRACT

Static lower_bound first? Or point lookup, scans and updates with fixed payloads?

2 · REPRODUCTIONS

Agree on pinned upstream implementations and the first real corpus.

3 · SUCCESS METRICS

Memory budget, exactness, lookup/scan cost, write tails and rebuild debt.

Next milestone: one reproducible common-data comparison, then iterate.

Appendix · one command for each evidence trail

Every lesson ends in a result JSON and an offline report section.

ID	Run / inspect	Classification
01	Rank fitting + corrected search	Teaching model
02	RadixSpline / binary exact lower_bound	Source-derived native
03	NFL table ratios + batching	Published evidence
04	Matched compression and routing	Experimental native map
05	Greedy / exhaustive virtual points	Teaching model
06	Geometry + matched mutation tails	Experimental native map
07	Archive / Mac evidence checks	Audit; no fresh timing

Appendix · rehearse once; keep a fallback open

Do not debug installations or download datasets during the meeting.

BEFORE THE MEETING

```
python3 walkthrough.py prepare  
python3 walkthrough.py demo all
```

LIVE: ONLY THREE COMMANDS

```
demo 02  
demo 05  
demo 06
```

IF A RUN FAILS

Open results/saved/report.html. Explain that these are recorded runs with their environment.

WHICH TAB?

Use this deck and the numbered result page. The older code guide is only a reference.

Appendix · source map, foundations

The user-provided NFL and CSV versions are preserved in sources/.

Source	Read for
Kraska et al. • SIGMOD 2018 • arXiv:1712.01208	RMI / learned key-to-position viewpoint
Galakatos et al. • SIGMOD 2019 • arXiv:1801.10207	FITing-Tree / bounded approximation
Ferragina & Vinciguerra • PVLDB 2020	PGM / recursive models and variants
Kipf et al. • aiDM 2020 • arXiv:2004.14541	RadixSpline / single-pass construction
Ding et al. • SIGMOD 2020 • microsoft/ALEX	ALEX / adaptive dynamic structure
Wu et al. • PVLDB 2021 • p1276-wu.pdf	LIPP / precise placement

Appendix · source map, transformation and compression

Keep each result attached to its own experimental contract.

Source	Read for
NFL • uploaded 2205.11807v1.pdf	Tables 1–2; batching and latency definitions
CSV • uploaded 2408.06134v3.pdf	Algorithms 1–2; promoted-key measurements
Boffa et al. • ALENEX 2021	LA-vector / learned compressed rank/select
LeCo • SIGMOD 2024 • arXiv:2306.15374	Serial correlations / random access
SEA 2025 • doi:10.4230/LIPIcs.SEA.2025.5	Strong static comparisons
RoBin • doi:10.1145/3749188; LINE • 3802080	Robustness / maintenance frontier